

Föreläsning 260512 – Samlingar (Collections) III

Nu ska vi fokusera på de generiska samlingarna:

Namespace: **System.Collections.Generic**

List<T>

Dictionary<TKey, TValue>

SortedDictionary<TKey, TValue>

Queue<T>

Stack<T>

SortedList<TKey, TValue>

Generics

Generics are a kind of parameter for a type, used to define what other type(s) it should refer to when it is used.

A good example of this is the List and Dictionary classes, that use generics to define which types of values they can store. They are defined like this in their class definitions:

The <T>, <TKey> and <TValue> parts of the class definitions means that they can be replaced with any kind of data type or class, when the actual instance of the class is created, like so:

```
List<int> listInt = new List<int>();  
Dictionary<int,string> dict = new Dictionary<int, string>();
```

Collections - Dictionaries

A Dictionary has much in common with a List – they are both single-dimensional collections of objects, they both use generics for their type.

However, a Dictionary uses two types – a Key type and a Value type.

The Value is the same as the type of a List – it is the actual value stored. `Dictionary<string, DateTime>`

The Key, meanwhile, is used as another identifier for that specific value, allowing you to use more complex values than a numeric index to reference a value:

```
dict["today"] = DateTime.Now;
```

Collections - List Initialization

Collections can be created and then assigned value, but much like variables they can also be initialized. This is done through object initialization:

```
List<int> listInt = new List<int> { 5,7,8,1,2,7,1 };
```

The **new** keyword creates the List object, and the code segment after it defines the content of the list. Dictionaries work the same way, but each element is its own list, with the Key and Value of each:

```
new Dictionary<string, string> { {"Eric", "eric.lindroth@lexicon.com"},  
                                {"Fredrik", "fredrik.odin@lexicon.com"},  
                                {"Kent", "kent.gudmundsen@lexicon.com"}  
};
```

Collections - *foreach* Loops

When dealing with collections, you often need to loop through all of the items contained inside them. While you can use a *while*- or *for*-loop to do this, you can also use a *foreach*-loop:

This kind of loop will take each item stored in the collection, in turn, assign it to a temporary variable, and run through the loop's code with it. After this, it will load the next value into the variable, and repeat, until all items have been looped through.

```
foreach (int item in listInt) {  
    Console.WriteLine(item);  
}
```

Vad innebär <T> ?

Vad betyder "generisk"?

Generisk = Kan ange parametrar med valfri typ: List<int>, List<string> osv.

Om vi jämför med en Array...

Array = Typen anges direkt:

// Ej generisk - typen hårdkodas

```
int[] arr = new int[5];           // Bara int!  
string[] str = new string[5];     // Bara string!
```

// Generisk - samma klass, olika typer

```
List<int> lista1 = new();           // int  
List<string> lista2 = new();       // string
```

Generiska datatyper är typer som använder en **typparameter**, oftast skriven som **T**, så att samma klass eller metod kan fungera med flera olika datatyper.

Detta betyder:

I stället för att skriva en separat klass för int, string eller en egen klass, skriver man en **mall** som till exempel **List<T>**.

Sedan kan man använda den som List<int>, List<string> eller List<Person>.

Varför använder man dessa typer?

- De gör koden återanvändbar.
- De ger typsäkerhet, så att fel typ upptäcks tidigare.
- De minskar behovet av att casta manuellt mellan typer.

Enkel regel:

Se generiska datatyper som en **mall**: man skriver logiken en gång och väljer datatyp när man använder den.

Deklaration av en generisk klass:

```
public class Lista<T>
{
    private T[] data;
    public void Add(T item) { /* ... */ }
}
```

Där **T** är en **typparameter**, och man använder den som Lista<int> eller Lista<string>.

Angående listor:

- It is dynamically sized, so if you add more elements into it, it will **resize itself** as needed.
- It has many built-in methods to deal with things like **finding items** within the list, **sorting** and **changing items**

Collections - Lists

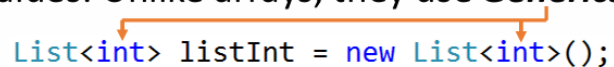
The List collection works much like an array does, but it also comes with its own conveniences added in:

It is **dynamically sized**, so if you add more elements into it, it will resize itself as needed.

It has many **built-in methods** to deal with things like finding items within the list, sorting and changing items.

As with arrays, it is defined for a specific object type, and can use indexes to refer to individual values. Unlike arrays, they use **Generics** to define their type.

```
List<int> listInt = new List<int>();
```



Collections - Lists - Useful members

Some of the useful members in the List collection include:

- | | |
|-------------|--|
| ▪ Add | Inserts a given item into the List |
| ▪ Remove | Removes a given item from the List |
| ▪ RemoveAt | Removes the item at a given index from the List |
| ▪ AddRange | Adds all the items in a given List to the List |
| ▪ FindIndex | Finds the index of a given item |
| ▪ Contains | Checks if a given item is in the list |
| ▪ Count | Returns the number of items that are in the list |
| ▪ Clear | Removes all items in the list |

```
using System.Collections.Generic;
```

```
namespace Generics
```

```
{
```

```
    internal class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            List<int> list = new List<int>();
```

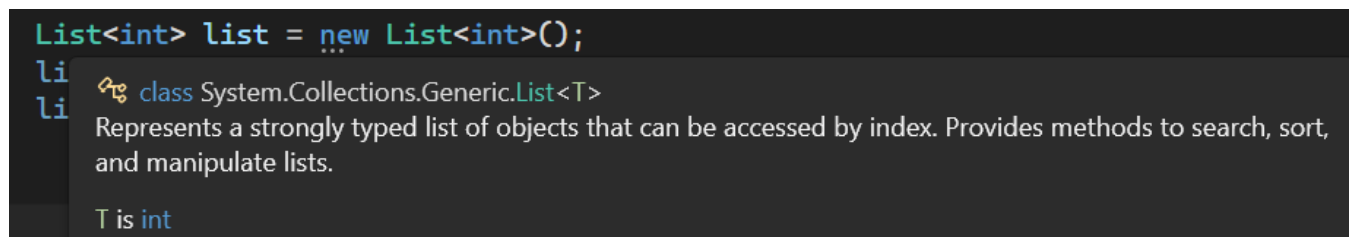
```
            list.Add(10);
```

```
            list.Add(20);
```

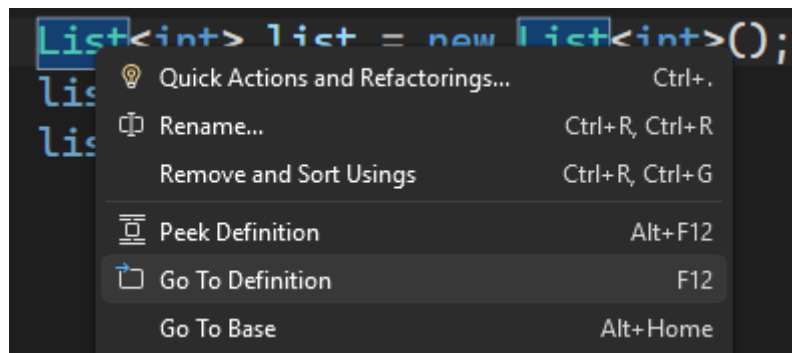
```
        }
```

```
    }
```

```
}
```



Om man högerklickar på List ovan och väljer "Go To Definition"



Ser vi hur listor är definierade.

"Implements a variable-size List that **uses an array of objects**"

IEnumerable<T>

Är ett grundläggande gränssnitt i .NET som möjliggör **iteration** över en sekvens av objekt av typen T, och det är nyckeln till foreach-loopar och LINQ-frågor.

Vad är IEnumerable<T> ?

Tänk på IEnumerable<T> som en "kontrakt" som säger: "Den här samlingen kan du loopa igenom en i taget".

Varför använda det?

Vi använder IEnumerable<T> för att ge **minimalistisk (minimal) åtkomst** till data – **bara iteration, inget indexering eller räkning**.

Vi gör ingenting med elementen utan bara loopar igenom, t.ex. vid utskrift m.m.

Det främjar **abstraktion** så att din kod fungerar med alla samlingar (List, Array, Dictionary) utan att bry sig om implementationen.

Det aktiverar också LINQ-metoder som Where, Select och deferred execution, där koden körs först vid iteration. Vi återkommer till detta senare.

När ska vi använda det?

Returtyp i metoder: Returnera IEnumerable<T> för flexibilitet, t.ex. i API:er.

Parametrar: Ta emot IEnumerable<T> för att acceptera olika samlingstyper.

LINQ och strömmar: För stora dataset eller streaming, då det inte laddar allt i minnet på en gång.

Använd `IEnumerable<T>` istället för `List<T>` när man vill hålla koden **flexibel, minnesvänlig** och **abstrakt** – det är en lämplig returtyp eller parameter för **read-only** scenarier.

När ska vi använda `IEnumerable<T>`?

för återigen **read-only åtkomst** och **deferred execution** (koden körs inte förrän man itererar).

Det passar väldigt bra när man:

Returnerar data från metoder (t.ex. repository eller LINQ-frågor) utan att exponera implementationen.

Hanterar stora dataset eller streaming (t.ex. från DB), då det inte laddar allt i minnet direkt.

När ska vi använda `List<T>`?

När vi behöver **modifiera, indexera** eller använda **Count**:

Lägga till/ta bort element (Add, Remove).

Snabb åtkomst via index (list[0]).

Hur kan vi använda IEnumerable i ett garage?

`IEnumerable<T>` är ett steg mot bättre design i ett garage-system, där man t.ex. hanterar bilar utan att ladda allt i minnet på en gång.

I vår garage-app (t.ex. om vi har inventering av bilar eller reparationer) ger `IEnumerable<T>` **lazy loading** och **abstraktion**:

Vi loopar eller filtrerar utan att använda en full Lista.

Skulle kunna användas för stora listor från databas eller filer, som "hämta reparationer från idag".